Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра интеллектуальных информационных технологий

Лабораторная работа № 6
По дисциплине: «Современные платформы программирования»
За 6 семестр

Выполнил:
Студент 3 курса
Группы ПО-12
Матюх И.М.
Проверил:
Кулик А.Д.

Брест 2026

Цель: освоить приемы тестирования кода на примере использования пакета pytest.

Вариант-3

Задание 1:

Написание тестов для мини-библиотеки покупок (shopping.py)

1. Создайте файл test_cart.py. Реализуйте следующие тесты:

• Проверка добавления товара: после add_item("Apple", 10.0) в корзине должен быть один элемент.

• Проверка выброса ошибки при отрицательной цене.

• Проверка вычисления общей стоимости (total()).

2. Протестируйте метод apply_discount с разными значениями скидки:

• 0% - цена остаётся прежней

• 50% - цена уменьшается вдвое

• 100% - цена становится ноль

• < 0% и > 100% - должно выбрасываться исключение

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Задание 3

Напишите метод String keep(String str, String pattern) который оставляет в первой строке все символы, которые присутствуют во второй.

Спецификация метода:

keep (None , None) = TypeError

keep (None, *) = None

keep ("", *) = ""

keep (* , None) = ""

keep (* , "") = ""

keep (" hello ", "hl") = " hll "

keep (" hello ", "le") = " ell "

Код программы

Test1.py

```
from unittest.mock import patch, ANY

import pytest
import requests


class Cart:
```

```
    def __init__(self):
        self.items = []

    def add_item(self, name: str, price: float):
        if price < 0:
            raise ValueError("Price cannot be negative")
        self.items.append({"name": name, "price": price})

    def total(self) -> float:
        return sum(item["price"] for item in self.items)

    def apply_discount(self, discount_percent: float):
        if discount_percent < 0 or discount_percent > 100:
            raise ValueError("Discount must be between 0 and 100")
        for item in self.items:
            item["price"] = item["price"] * (1 - discount_percent / 100)

    def apply_coupon(self, coupon_code: str):
        coupons = {"SAVE10": 10, "HALF": 50}
        if coupon_code in coupons:
            self.apply_discount(coupons[coupon_code])
        else:
            raise ValueError("Invalid coupon")


def log_purchase(item):
    requests.post("https://example.com/log", json=item, timeout=5)


class TestCart:
    def test_add_item(self, cart):
        cart.add_item("Apple", 10.0)
        assert len(cart.items) == 1

    def test_add_item_negative_price_raises_error(self, cart):
        with pytest.raises(ValueError, match="Price cannot be negative"):
            cart.add_item("Apple", -10.0)

    def test_total(self, cart):
        cart.add_item("Apple", 10.0)
        cart.add_item("Banana", 5.0)
        assert cart.total() == 15.0

    @pytest.mark.parametrize(
        "discount,expected_price",
        [
```

```
            (0, 100.0),
            (50, 50.0),
            (100, 0.0),
        ],
    )
    def test_apply_discount(self, cart, discount, expected_price):
        cart.add_item("Item", 100.0)
        cart.apply_discount(discount)
        assert cart.items[0]["price"] == expected_price

    @pytest.mark.parametrize("invalid_discount", [-1, 101])
    def test_apply_discount_invalid_raises_error(self, cart, invalid_discount):
        cart.add_item("Item", 100.0)
        with pytest.raises(ValueError, match="Discount must be between 0 and 100"):
            cart.apply_discount(invalid_discount)


class TestLogPurchase:
    @patch("requests.post")
    def test_log_purchase_calls_post_with_correct_data(self, mock_post):
        item = {"name": "Apple", "price": 10.0}
        log_purchase(item)
        mock_post.assert_called_once_with(
            "https://example.com/log", json=item, timeout=ANY
        )


class TestApplyCoupon:
    @pytest.mark.parametrize(
        "coupon,discount",
        [
            ("SAVE10", 10),
            ("HALF", 50),
        ],
    )
    def test_apply_coupon_valid(self, cart, coupon, discount):
        cart.add_item("Item", 100.0)
        cart.apply_coupon(coupon)
        assert cart.items[0]["price"] == 100.0 * (1 - discount / 100)

    def test_apply_coupon_invalid_raises_error(self, cart):
        cart.add_item("Item", 100.0)
        with pytest.raises(ValueError, match="Invalid coupon"):
            cart.apply_coupon("INVALID")

    def test_apply_coupon_with_monkeypatch_coupons_dict(self, cart, monkeypatch):
```

```
        def patched_apply_coupon(self, coupon_code):
            test_coupons = {"TEST": 25}
            if coupon_code in test_coupons:
                self.apply_discount(test_coupons[coupon_code])
            else:
                raise ValueError("Invalid coupon")

        monkeypatch.setattr(Cart, "apply_coupon", patched_apply_coupon)
        cart.add_item("Item", 100.0)
        cart.apply_coupon("TEST")
        assert cart.items[0]["price"] == 75.0


@pytest.fixture
def cart():
    return Cart()


if __name__ == "__main__":
    pytest.main([__file__, "-v"])
```

Test2.py

```
import pytest


def rep(start, end, step):
    if start >= end:
        raise ValueError("start должно быть меньше end")
    result = []
    current = start
    while current <= end:
        result.append(current)
        current += step
    return result


def is_palindrome(s: str) -> bool:
    cleaned = "".join(ch.lower() for ch in s if ch.isalnum())
    return cleaned == cleaned[::-1]


def test_rep_basic():
    assert rep(1, 5, 1) == [1, 2, 3, 4, 5]


def test_rep_step_2():
```

```
    assert rep(1, 10, 2) == [1, 3, 5, 7, 9]


def test_rep_error_start_greater():
    with pytest.raises(ValueError, match="start должно быть меньше end"):
        rep(5, 1, 1)


def test_palindrome_word():
    assert is_palindrome("radar") == True


def test_palindrome_phrase():
    assert is_palindrome("A man a plan a canal panama") == True


def test_non_palindrome():
    assert is_palindrome("hello") == False


def test_palindrome_with_punctuation():
    assert is_palindrome("Was it a car or a cat I saw?") == True


def test_palindrome_mixed_case():
    assert is_palindrome("RaceCar") == True


if __name__ == "__main__":
    pytest.main([__file__, "-v"])
```

Test3.py

```
import pytest


def keep(s: str, pattern: str) -> str:
    if s is None and pattern is None:
        raise TypeError("Both arguments cannot be None")

    if s is None:
        return None
    if pattern is None:
        return ""
    if s == "" or pattern == "":
        return ""
```

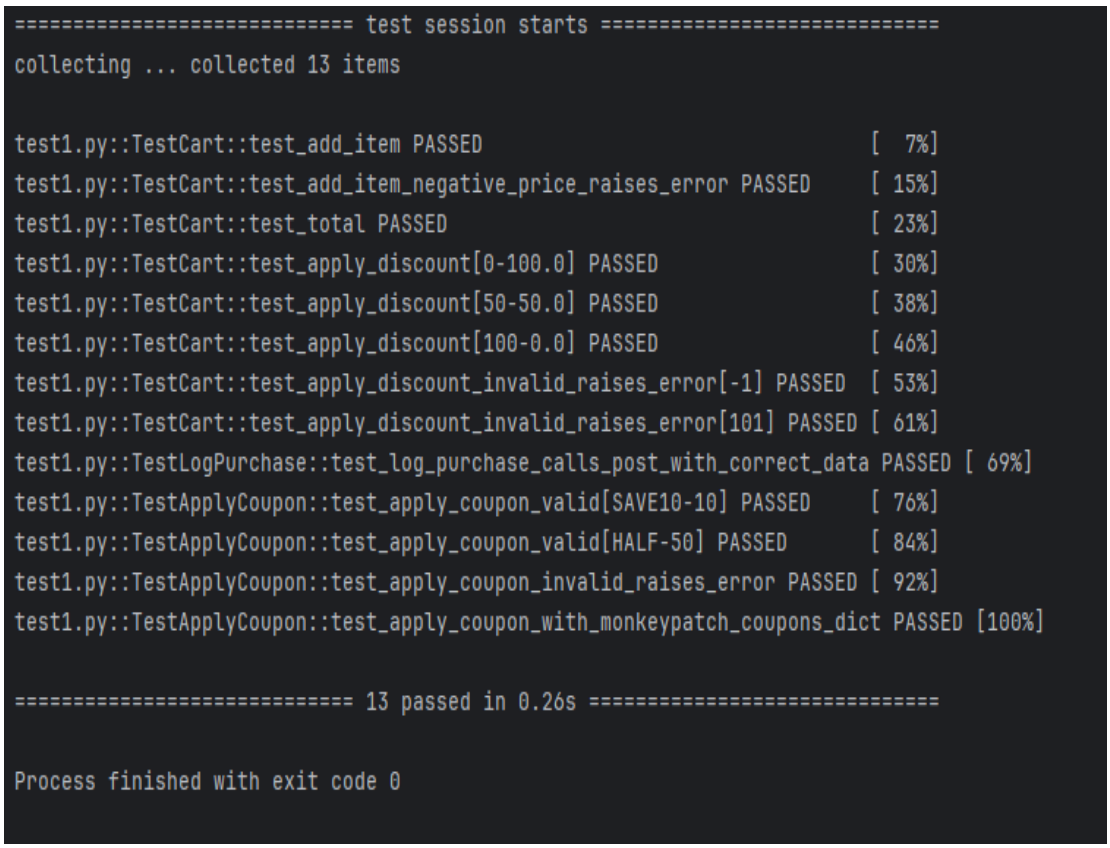
```
    result = ""
    for char in s:
        if char in pattern:
            result += char

    return result


class TestKeep:
    def test_both_none_raises_type_error(self):
        with pytest.raises(TypeError):
            keep(None, None)

    def test_first_none_returns_none(self):
        assert keep(None, "abc") is None

    def test_first_empty_returns_empty(self):
        assert keep("", "abc") == ""

    def test_second_none_returns_empty(self):
        assert keep("abc", None) == ""

    def test_second_empty_returns_empty(self):
        assert keep("abc", "") == ""

    def test_keep_hl(self):
        assert keep(" hello ", "hl") == "hll"

    def test_keep_le(self):
        assert keep(" hello ", "le") == "ell"

    def test_keep_no_match(self):
        assert keep("abc", "xyz") == ""

    def test_keep_all_match(self):
        assert keep("aaa", "a") == "aaa"

    def test_pattern_longer_than_string(self):
        assert keep("ab", "abcd") == "ab"

    def test_keep_with_duplicates_in_pattern(self):
        result = keep("hello", "ll")
        assert result == "ll"
```

```
if __name__ == "__main__":
    pytest.main([__file__, "-v"])
```

Результат выполнения программы:

```
============================= test session starts ==============================
collecting ... collected 8 items

test2.py::test_rep_basic PASSED                                          [ 12%]
test2.py::test_rep_step_2 PASSED                                         [ 25%]
test2.py::test_rep_error_start_greater PASSED                            [ 37%]
test2.py::test_palindrome_word PASSED                                    [ 50%]
test2.py::test_palindrome_phrase PASSED                                  [ 62%]
test2.py::test_non_palindrome PASSED                                     [ 75%]
test2.py::test_palindrome_with_punctuation PASSED                        [ 87%]
test2.py::test_palindrome_mixed_case PASSED                              [100%]

============================== 8 passed in 0.02s ===============================

Process finished with exit code 0
```

```
============================= test session starts ==============================
collecting ... collected 11 items

test3.py::TestKeep::test_both_none_raises_type_error PASSED              [  9%]
test3.py::TestKeep::test_first_none_returns_none PASSED                  [ 18%]
test3.py::TestKeep::test_first_empty_returns_empty PASSED                [ 27%]
test3.py::TestKeep::test_second_none_returns_empty PASSED                [ 36%]
test3.py::TestKeep::test_second_empty_returns_empty PASSED               [ 45%]
test3.py::TestKeep::test_keep_hl PASSED                                  [ 54%]
test3.py::TestKeep::test_keep_le PASSED                                  [ 63%]
test3.py::TestKeep::test_keep_no_match PASSED                            [ 72%]
test3.py::TestKeep::test_keep_all_match PASSED                           [ 81%]
test3.py::TestKeep::test_pattern_longer_than_string PASSED               [ 90%]
test3.py::TestKeep::test_keep_with_duplicates_in_pattern PASSED          [100%]

============================== 11 passed in 0.03s ==============================

Process finished with exit code 0
```



Вывод: освоил приемы тестирования кода на примере использования пакета pytest.